

Product Requirements Document (PRD)

Product: Digital Tipping & Venue Settlement Platform

Working name: TBD

Market: Nigeria

Initial market: Lagos nightlife, clubs, lounges, concerts, and private events

Document status: Product Definition / MVP

Version: 1.0

Date: September 2026

1. Executive Summary

The product is a digital tipping and payment infrastructure designed for Nigeria's nightlife and entertainment industry.

It allows guests at clubs, lounges, concerts, and events to financially support entertainers—such as DJs, dancers, MCs, musicians, and performers—without needing cash, knowing their bank details, downloading an application, or creating an account.

A guest scans a QR code or taps an NFC tag associated with an entertainer.

They see:

Tip DJ Mike

They select an amount, choose whether their name should be displayed, complete payment through a licensed payment processor, and receive a confirmation.

Behind the scenes, the platform:

1. Identifies the venue and entertainer.
2. Creates and records the payment.
3. Attributes the payment to the correct entertainer.
4. Applies the venue's configured split rules.
5. Records the transaction in an immutable double-entry ledger.
6. Initiates the entertainer's payout.
7. Records the payout and settlement.
8. Provides the venue with a transparent financial dashboard.

The initial product is intentionally simple.

The guest does not need an app.

The entertainer does not need to manually request payment.

The venue does not need to reconcile cash sprayed or handed directly to performers.

The platform creates a **digital, traceable, auditable flow of money around physical entertainment.**

2. Product Vision

Vision

Build the financial infrastructure for live entertainment in Africa.

The initial product is digital tipping.

The long-term opportunity is much larger:

A programmable payment, attribution, split, and settlement layer for physical entertainment and events.

The system should eventually allow payment providers, fintechs, banks, venues, event platforms, and other businesses to use the platform's payment attribution and settlement infrastructure through APIs.

3. Problem

Nigeria has rapidly adopted digital payments, but entertainment environments still rely heavily on cash and informal payment flows.

In nightlife environments, money can move between:

- guests
- entertainers
- DJs
- dancers
- MCs
- musicians
- venues
- event organizers
- promoters

without a reliable digital record.

This creates several problems.

Guests

- May not have cash.
- May not know an entertainer's bank details.
- May not want to leave the club to find an ATM.

- May want to tip quickly without downloading an application.
- May want recognition for their contribution.
- May want to remain anonymous.

Entertainers

- Depend on physical cash.
- Can lose or misplace cash.
- Have limited visibility into total tips received.
- May have difficulty proving how much they received.
- May wait for cash settlement.
- Have no standardized digital tipping identity.

Venues

- Have limited visibility into tipping activity.
- May have difficulty reconciling money.
- May have disputes between entertainers and management.
- Have no standardized tipping infrastructure.
- Carry reputational and operational risk around uncontrolled cash activity.

Event organizers

- Need a simple mechanism for audience-to-performer payments.
- Need transparent settlement.
- May need to support multiple performers.

4. Product Opportunity

The product converts:

Physical tipping behavior → digital payment → attribution → ledger → settlement

without requiring the guest to change their behavior dramatically.

The product should feel less like a banking application and more like:

Scan → choose amount → pay → done.

The complexity belongs to the infrastructure, not the customer.

5. Product Principles

5.1 Zero friction

The guest should never need to:

- create an account
- download an app
- remember a password
- manually enter a bank account
- search for an entertainer

5.2 Financial correctness over feature quantity

The ledger must be correct before the product becomes feature-rich.

5.3 Processor agnostic

The platform should be able to work with multiple payment providers over time.

5.4 Transparent money movement

Every transaction should be traceable from:

payment → ledger → allocation → payout → settlement.

5.5 Privacy by default

Guests choose whether their identity is displayed.

5.6 Mobile-first

The guest experience is designed primarily for smartphones.

5.7 Infrastructure first

The consumer experience is the entry point.

The long-term asset is the underlying financial infrastructure.

6. Target Market

Initial market

Lagos nightlife and entertainment.

Potential venues:

- nightclubs
- lounges
- bars
- concert venues
- event centers
- private event organizers

Initial entertainers

- DJs
 - dancers
 - MCs
 - live bands
 - singers
 - performers
-

7. Personas

7.1 Guest

A person attending a club, concert, event, or lounge.

Goals

- Tip quickly.
- Avoid cash.
- Support a performer.
- Optionally receive recognition.
- Get confirmation of payment.

Pain points

- No cash.
 - Doesn't know bank details.
 - Payment friction.
 - Doesn't want another app.
-

7.2 Entertainer

A DJ, dancer, MC, musician, or performer.

Goals

- Receive tips.
- Get paid quickly.

- Know exactly how much was received.
- Have a transaction history.
- Withdraw funds to a bank account.

Pain points

- Cash handling.
 - Lost payments.
 - Lack of records.
 - Settlement disputes.
-

7.3 Venue Manager

Responsible for managing a club/lounge/event.

Goals

- Manage entertainers.
- Configure payment rules.
- See transaction activity.
- Monitor settlement.
- Reduce cash-related operational issues.

Pain points

- Reconciliation.
 - Disputes.
 - Lack of visibility.
 - Manual processes.
-

7.4 Platform Administrator

Internal operations team.

Goals

- Manage venues.
 - Manage entertainers.
 - Monitor payments.
 - Resolve disputes.
 - Monitor failed payouts.
 - Reconcile provider balances.
-

8. Core Product

The MVP consists of five major systems.

1. Guest tipping experience
 2. Venue management
 3. Entertainer management
 4. Payment + ledger infrastructure
 5. Payout + settlement infrastructure
-

9. Guest Experience

Entry

Guest scans a QR code or taps an NFC tag.

Example:

```
yourproduct.com/t/8xK29Lm
```

The backend resolves the token to:

```
Venue: Club Example  
Entertainer: DJ Mike  
Location: Main Floor
```

10. Guest Tipping Screen

The screen displays:

```
DJ MIKE
```

```
Show some love ❤️
```

```
₦1,000
```

```
₦2,000
```

```
₦5,000
```

```
₦10,000
```

```
[ Custom Amount ]
```

```
☐ Anonymous
```

```
☒ Show my name
```

[Continue]

The page should:

- load quickly
- work without login
- work without app installation
- be mobile-first
- default to a dark theme
- clearly identify the entertainer
- clearly identify the venue where appropriate

11. Amount Selection

Preset amounts should be displayed prominently.

Default:

- ₦1,000
- ₦2,000
- ₦5,000
- ₦10,000

The guest may enter a custom amount.

Requirements:

- minimum configurable tip
- maximum configurable tip
- integer currency values
- no floating-point monetary calculations

12. Guest Identity

The guest can select:

Anonymous

The venue may see:

Anonymous — ₦5,000

Show my name

The venue may see:

Quayyum — ₦5,000

The identity preference must be explicitly chosen.

The platform should not expose unnecessary guest information to entertainers or other guests.

13. Payment

The platform should not collect or store raw card information.

The guest is redirected or presented with the payment provider's secure checkout.

Initial provider:

Paystack

Potential future providers:

- Flutterwave
- other licensed payment providers

Supported payment methods depend on the selected provider and current availability.

Initial target:

- bank transfer
 - card
-

14. Payment Success

After successful payment:

Payment successful

₦5,000

You tipped DJ Mike.

Reference:
TIP-8F92KX

✓ Payment confirmed

If spectacle mode is enabled:

Your tip is now showing on the venue screen.

15. Payment Failure

The guest should receive a simple message:

```
Payment unsuccessful

Your payment wasn't completed.

[ Try Again ]
```

The system must distinguish:

- payment failed
 - payment abandoned
 - payment pending
 - payment successful
 - payment reversed/refunded
-

16. QR/NFC Architecture

Each QR/NFC identifier maps to a platform-controlled token.

Example:

```
QR Token
  ↓
Venue
  ↓
Entertainer
  ↓
Location
```

The token should not directly expose internal database IDs.

Example:

```
/t/{public_token}
```

QR codes must support:

- activation

- deactivation
 - replacement
 - reassignment where appropriate
 - audit history
-

17. Venue System

Venue administrators can:

Manage venue

- name
- logo
- location
- settings
- settlement configuration

Manage entertainers

- add entertainer
- remove entertainer
- activate/deactivate entertainer
- view payout status

Manage QR codes

- generate
- download
- deactivate
- regenerate

Configure payment rules

Example:

```
Entertainer: 85%  
Venue: 10%  
Platform: 5%
```

18. Entertainer System

Each entertainer has a profile.

```
Stage Name  
Legal Name
```

Phone
Bank
Account Number
KYC Status
Payout Status
Venue

The entertainer should eventually have a dashboard showing:

Tonight
₦184,000

This Week
₦421,500

Total
₦2,140,000

Transaction history:

₦5,000	11:42 PM	Paid
₦10,000	11:39 PM	Paid
₦2,000	11:31 PM	Paid

19. KYC

Entertainers receiving payouts must complete the applicable verification process required by the platform's payment/compliance structure.

The onboarding flow should support:

Enter bank details
↓
Resolve account
↓
Confirm account holder
↓
Identity verification
↓
Verified

KYC statuses:

NOT_STARTED
PENDING
VERIFIED
FAILED
REVIEW
SUSPENDED

The system must not unnecessarily store sensitive identity information.

20. Financial Ledger

The ledger is the financial source of truth.

Every monetary movement must produce balanced accounting entries.

Example:

Guest tips:

¥5,000

Split:

- Entertainer: 85%
- Venue: 10%
- Platform: 5%

Ledger:

Processor Clearing
DEBIT ¥5,000

Entertainer Payable
CREDIT ¥4,250

Venue Payable
CREDIT ¥500

Platform Revenue
CREDIT ¥250

Total:

```
Debit  = ₦5,000
Credit = ₦5,000
```

The ledger must always balance.

21. Money Representation

All monetary values must be stored as integers in the smallest currency unit.

For Nigerian naira:

```
₦5,000
=
500000 kobo
```

Never use floating-point numbers for financial calculations.

22. Core Database Entities

Minimum entities:

```
users
venues
entertainers
venue_entertainers
qr_codes
guest_sessions
payment_transactions
ledger_accounts
ledger_entries
split_rules
payout_accounts
payouts
```

```
webhook_events  
reconciliation_runs
```

23. Payment Transaction

Conceptual structure:

```
payment_transactions  
  
id  
external_reference  
provider  
venue_id  
entertainer_id  
guest_session_id  
gross_amount_kobo  
currency  
guest_display_name  
display_name_enabled  
status  
created_at  
updated_at
```

`external_reference` must be unique.

This protects against duplicate processing.

24. Ledger Account

Types:

```
PROCESSOR_CLEARING  
ENTERTAINER_PAYABLE  
VENUE_PAYABLE  
PLATFORM_REVENUE
```

Accounts should be created per relevant entity where necessary.

Example:

DJ Mike Payable
Club Example Payable
Platform Revenue
Processor Clearing

25. Ledger Entry

```
ledger_entries

id
transaction_id
account_id
direction
amount_kobo
created_at
```

Allowed directions:

DEBIT
CREDIT

Ledger entries should be immutable.

Corrections should be represented by additional compensating entries rather than editing historical entries.

26. Split Engine

The split engine calculates how a payment is allocated.

Input:

```
Gross = ₦5,000

Entertainer = 85%
Venue = 10%
Platform = 5%
```

Output:

Entertainer = ₦4,250
Venue = ₦500
Platform = ₦250

The split configuration must be versioned so historical transactions retain the rule that was active when the payment occurred.

27. Payment Lifecycle

Payment states:

CREATED
PENDING
SUCCESS
FAILED
ABANDONED
REVERSED
REFUNDED

Ledger processing should only occur after the payment has been independently confirmed as successful.

28. Webhook Processing

Payment provider webhooks must:

1. Receive event.
2. Verify signature.
3. Store raw event.
4. Check idempotency.
5. Queue processing.
6. Verify transaction.
7. Create ledger entries.
8. Trigger payout where appropriate.
9. Record processing result.

The system must safely handle duplicate webhook events.

29. Raw Webhook Event Log

Every provider event should be preserved.

Example:

```
webhook_events

id
provider
event_type
event_id
signature
payload
received_at
processed_at
processing_status
```

This enables:

- auditing
- debugging
- replay
- dispute resolution

30. Payout System

After the ledger creates an entertainer payable balance:

```
Entertainer Payable
    ↓
Payout Queue
    ↓
Payout Service
    ↓
Bank Transfer
    ↓
Payout Confirmation
```

Payout states:

```
QUEUED
PROCESSING
SUCCESS
FAILED
RETRYING
CANCELLED
```

Payouts must be idempotent.

31. Failed Payouts

If a payout fails:

Payment
↓
Ledger
↓
Entertainer payable
↓
Payout failed

The money remains owed to the entertainer.

It must never disappear because a bank transfer failed.

The system should support:

- retry
- account correction
- manual review
- payout cancellation where appropriate

32. Venue Settlement

Venue settlement should be separately tracked.

The venue may have:

Venue Payable

which is settled according to the agreed commercial arrangement.

The system should distinguish:

Entertainer payout

from

Venue settlement

from

33. Reconciliation

A reconciliation process compares:

```
Internal Ledger
    ↓
Payment Provider
    ↓
Payout Provider/Bank
```

The system should identify:

- missing payments
- duplicate payments
- unmatched provider transactions
- failed payouts
- unexpected balances
- reversed payments
- refunds
- settlement differences

At MVP scale, reconciliation can run hourly or daily.

34. Venue Dashboard

The dashboard should provide:

Overview

Tonight

Total Tips
₦537,500

Transactions
183

Entertainers
7

Pending Payouts
₦21,000

Entertainers

DJ Mike	₦184,000
MC Tunde	₦72,000
Sarah	₦51,500
Live Band	₦230,000

Transactions

Time
Amount
Entertainer
Guest
Status
Reference

Payouts

Entertainer
Amount
Status
Reference
Time

35. Spectacle Mode

Spectacle mode is an optional venue feature.

The venue can display live tipping activity on a screen.

Example:

TONIGHT

🔥 DJ MIKE

₦10,000
from Anonymous

₦5,000
from Tunde

¥2,000
from Sarah

The venue can configure:

Show names
ON/OFF

Show amounts
ON/OFF

Show individual tips
ON/OFF

Leaderboard
ON/OFF

The guest's display preference must always be respected.

36. Quiet Mode

Quiet mode provides no public display.

The guest receives only:

Payment successful
¥5,000

This mode is suitable for:

- weddings
- corporate events
- private events
- restaurants
- lounges
- low-key venues

37. Notifications

MVP notifications:

Guest

- payment success

- payment failure

Entertainer

- tip received
- payout initiated
- payout completed
- payout failed

Venue

- settlement
- reconciliation issue
- operational alerts

Channels may eventually include:

- SMS
 - email
 - WhatsApp
 - push notifications
-

38. API Architecture

The initial backend will be a modular NestJS monolith.

Core modules:

```
auth
venues
entertainers
qr
payments
ledger
splits
kyc
payouts
webhooks
reconciliation
notifications
admin
```

The system should expose REST APIs initially.

39. Technology Stack

Backend

```
NestJS  
TypeScript  
PostgreSQL  
Prisma  
Redis  
BullMQ
```

Frontend

```
React / Next.js  
TypeScript  
Tailwind CSS
```

Infrastructure

```
Docker  
CI/CD  
Cloud deployment  
Managed PostgreSQL  
Managed Redis
```

Payment

Initial:

```
Paystack
```

Architecture must allow future payment-provider adapters.

40. Payment Provider Abstraction

The domain should not depend directly on Paystack.

Conceptually:

```
interface PaymentProvider {  
  initializePayment(): Promise<Payment>;  
}
```

```
verifyPayment(): Promise<Payment>;
verifyWebhook(): boolean;
}
```

Payout:

```
interface PayoutProvider {
  createRecipient(): Promise<Recipient>;
  initiateTransfer(): Promise<Transfer>;
  verifyTransfer(): Promise<Transfer>;
}
```

Paystack becomes one implementation.

41. Security Requirements

The platform handles financial transactions and therefore requires strong security controls.

Requirements:

- HTTPS everywhere
 - webhook signature verification
 - encrypted secrets
 - role-based access control
 - audit logs
 - database constraints
 - idempotency
 - rate limiting
 - input validation
 - secure authentication
 - least-privilege access
 - encrypted sensitive data where appropriate
 - no storage of raw card information
 - secure payment-provider integration
-

42. Roles and Permissions

Platform Admin

Full platform access.

Venue Admin

Access only to their venue.

Entertainer

Access only to their own financial information.

Guest

No account required.

Guest access is limited to the current tipping transaction.

43. MVP Scope

The first production MVP must contain:

Guest

- QR scan
- entertainer page
- preset amounts
- custom amount
- anonymous/name choice
- payment
- success page

Venue

- venue account
- entertainer management
- QR generation
- split configuration
- transaction dashboard

Entertainer

- onboarding
- bank account
- KYC status
- transaction history
- payout history

Platform

- payment processing
 - webhook processing
 - double-entry ledger
 - payout system
 - reconciliation
 - audit trail
-

44. Explicitly Out of Scope for MVP

Do not build:

- guest mobile app
 - guest accounts
 - consumer wallet
 - cryptocurrency
 - loyalty points
 - social feed
 - AI features
 - complex gamification
 - multi-country support
 - multiple payment processors initially
 - microservice architecture
 - advanced analytics
 - marketplace
 - lending
 - credit
 - stored-value wallet
-

45. MVP User Journey

Guest

```
See QR
↓
Scan
↓
Entertainer page
↓
Choose amount
↓
Choose identity visibility
↓
Payment checkout
↓
Payment success
↓
Receipt
```

Entertainer

```
Venue adds entertainer
↓
```

```
Complete onboarding
↓
Bank account verified
↓
KYC completed
↓
QR activated
↓
Receive tips
↓
Payout initiated
↓
Money reaches bank
```

Venue

```
Create venue
↓
Add entertainers
↓
Configure splits
↓
Generate QR codes
↓
Deploy QR/NFC tags
↓
Monitor tips
↓
View settlement
```

46. End-to-End Transaction

Example:

Guest tips DJ Mike:

₦5,000

Step 1

Guest scans QR.

Step 2

Platform resolves:

Venue = Club X
Entertainer = DJ Mike

Step 3

Payment initialized.

Step 4

Guest pays ₦5,000.

Step 5

Provider sends webhook.

Step 6

Webhook signature verified.

Step 7

Duplicate reference checked.

Step 8

Payment confirmed.

Step 9

Split calculated:

DJ	₦4,250
Venue	₦500
Platform	₦250

Step 10

Ledger entries created atomically.

Step 11

Entertainer payout queued.

Step 12

Transfer initiated.

Step 13

Transfer confirmed.

Step 14

Payout ledger entries recorded.

Step 15

Entertainer sees payout.

Step 16

Venue dashboard updates.

47. Error Scenarios

The system must handle:

Duplicate webhook

Do not process twice.

Payment successful but webhook delayed

Payment remains pending until verified.

Payout fails

Keep entertainer payable balance.

Bank account invalid

Block/retry payout without losing funds.

Provider unavailable

Queue operation and retry.

Database failure

Atomic transactions prevent partial ledger writes.

QR deactivated

Guest sees:

This tipping link is no longer active.

Entertainer removed

Existing financial records remain immutable.

48. Business Model

The platform can generate revenue through a small platform fee.

Example:

Tip = ₦5,000

Platform fee = 5%

Platform revenue = ₦250

Potential commercial models:

Model A — Transaction fee

Percentage of each tip.

Model B — Venue subscription

Monthly SaaS fee plus lower transaction fee.

Model C — Hybrid

Monthly venue fee + transaction fee.

Model D — Infrastructure/API

Long-term:

Charge businesses for payment splitting and settlement infrastructure.

The MVP should test transaction-based pricing first.

49. Go-To-Market Strategy

The initial strategy should be **venue-first and entertainer-assisted**.

Target:

2–3 Lagos venues
+
5–15 entertainers
+
real paying guests

Start with venues where:

- DJs are popular
- performers already receive tips
- cash is common
- management is receptive to technology
- there is strong social activity

50. Pilot Strategy

Pilot 1

One venue.

Approximately:

1 venue
3–5 entertainers
1 weekend

Measure:

- QR scans
- payment conversion
- total tips
- average tip
- payout success
- failed transactions
- guest feedback
- entertainer feedback

51. Pilot 2

Expand to:

3 venues
10–20 entertainers

Compare:

- venue type
- performer type
- tip size
- payment method
- conversion
- retention

52. Acquisition Strategy

The most important acquisition channel is the entertainer.

Instead of only selling:

“We have a digital tipping platform.”

Sell:

“Your fans can tip you instantly without cash or asking for your account number.”

Then use entertainers to pull venues into the platform.

Venue pitch:

Give your entertainers a safer, trackable way to receive tips while giving your venue visibility into the money flowing through your floor.

53. Key Product Metrics

North Star Metric

Successful tips processed through the platform.

Supporting metrics:

Acquisition

- venues onboarded
- entertainers onboarded
- QR codes activated

Engagement

- QR scans
- checkout starts
- completed payments

Conversion

completed payments

÷

QR scans

Financial

- GMV
- average tip
- platform revenue
- payout volume
- payout success rate

Reliability

- webhook success
- duplicate processing rate
- reconciliation discrepancies
- payout failure rate

54. MVP Success Criteria

The MVP is successful if:

Product

- guest can complete a tip in under one minute
- no guest account is required
- payment works reliably
- QR resolution works reliably

Financial

- every successful payment has balanced ledger entries
- duplicate webhooks never create duplicate money
- every payout has a traceable reference
- reconciliation produces zero unexplained discrepancies

Business

- at least 2–3 venues agree to pilot
- entertainers actively use the system

- real guests complete transactions
 - venues see enough value to continue using it
-

55. Performance Requirements

Guest page:

Target initial load under 2 seconds on a reasonable mobile connection.

API:

Target p95 response under 500ms for normal non-payment requests.

Webhook:

Acknowledge quickly and process asynchronously.

The payment workflow must not depend on a long-running HTTP request.

56. Reliability Requirements

Financial operations must be:

- atomic
- idempotent
- auditable
- retryable
- observable

Critical operations must never rely on:

“It probably worked.”

57. Observability

Track:

- API errors
- payment failures
- webhook failures
- payout failures
- queue failures
- reconciliation discrepancies
- provider latency

- database errors

Critical financial anomalies should trigger alerts.

58. Auditability

The platform should answer:

Who paid?

How much?

When?

For whom?

At which venue?

What split rule applied?

Where did the money go?

Which provider transaction processed it?

Which payout transferred it?

Was it refunded?

Was it reconciled?

The system should be able to reconstruct the entire lifecycle of a transaction.

59. Compliance & Legal

The platform operates around regulated financial activity.

Before production launch, the company must establish the appropriate legal and regulatory structure for its exact operating model.

Areas requiring professional/legal/compliance review include:

- payment processing
- payment collection
- funds custody
- payout structure
- KYC

- AML/CFT
- data protection
- consumer protection
- refunds
- chargebacks
- transaction records
- tax
- licensing/partnership requirements

The MVP architecture should therefore avoid unnecessarily becoming a wallet or holding customer funds independently.

The platform should initially operate through appropriate licensed payment providers and partners while the legal structure is established.

“Instant payout” must be treated as a product target until the actual provider, settlement, compliance, and transfer mechanics have been validated.

60. Development Roadmap

Phase 0 — Product & Compliance

1–2 weeks

Deliverables:

- final product specification
 - payment-provider validation
 - legal/compliance review
 - commercial model
 - pilot venue identification
-

Phase 1 — Foundation

1 week

Build:

- NestJS
- PostgreSQL
- Prisma
- Redis
- BullMQ
- authentication
- logging
- error handling
- CI/CD

- environment management
-

Phase 2 — Core Domain

1-2 weeks

Build:

- venues
 - entertainers
 - QR codes
 - split rules
 - guest sessions
 - database relationships
-

Phase 3 — Payment

1 week

Build:

- payment initialization
 - payment verification
 - provider adapter
 - payment states
 - checkout flow
-

Phase 4 — Ledger

1-2 weeks

Build:

- ledger accounts
- ledger entries
- split engine
- atomic transactions
- idempotency
- immutable event records

This phase receives the highest testing priority.

Phase 5 — Webhooks

1 week

Build:

- webhook endpoint
 - signature verification
 - raw event storage
 - event queue
 - duplicate handling
 - replay mechanism
-

Phase 6 — KYC + Payouts

1-2 weeks

Build:

- entertainer onboarding
 - bank account verification
 - KYC state
 - payout provider
 - payout queue
 - retries
 - payout reconciliation
-

Phase 7 — Dashboards

1-2 weeks

Build:

- venue dashboard
 - entertainer dashboard
 - transaction history
 - payout history
 - settlement view
-

Phase 8 — Pilot

2-4 weeks

Launch with:

2–3 venues
5–20 entertainers

Observe real-world behavior.

Fix:

- payment friction
- QR problems
- payout issues
- onboarding
- venue workflow
- guest UX

61. Future Roadmap

Version 2

- NFC
- WhatsApp notifications
- SMS
- better venue analytics
- live spectacle screens
- tipping leaderboard
- entertainer profiles
- event management

Version 3

- multiple payment processors
- multiple payout providers
- event organizers
- corporate events
- weddings
- concerts
- API access

Version 4

Open the infrastructure.

Third parties can use:

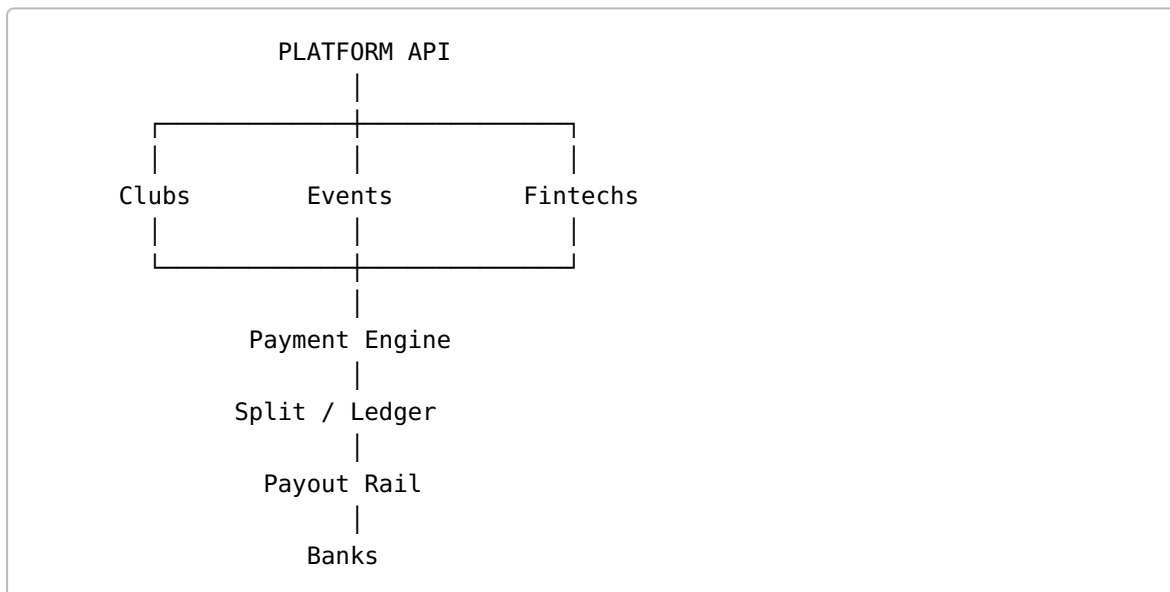
Payment
Attribution
Splitting

Ledger
Payout

through APIs.

62. Long-Term Platform

The long-term architecture becomes:



The QR tipping product is therefore not necessarily the final company.

It is the **first application built on the infrastructure.**

63. Product Moat

The QR code itself is not the moat.

The moat becomes:

1. Transaction history

The platform develops a trusted financial record of entertainment payments.

2. Venue network

More venues increase distribution.

3. Entertainer network

Entertainers can potentially carry their tipping identity across venues.

4. Payment infrastructure

The split and settlement engine becomes reusable.

5. Operational knowledge

The platform learns how money moves through entertainment environments.

6. Integrations

Payment providers, banks, event platforms, and venues become increasingly integrated.

64. Fundamental Product Loop

The entire MVP should be reducible to:

```
graph TD; A[SCAN] --> B[IDENTIFY]; B --> C[CHOOSE AMOUNT]; C --> D[PAY]; D --> E[CONFIRM]; E --> F[RECORD]; F --> G[SPLIT]; G --> H[PAYOUT]; H --> I[RECONCILE];
```

SCAN
↓
IDENTIFY
↓
CHOOSE AMOUNT
↓
PAY
↓
CONFIRM
↓
RECORD
↓
SPLIT
↓
PAYOUT
↓
RECONCILE

That is the product.

Everything else is secondary.

65. Definition of Done

The MVP is considered production-ready when:

- [] Venue can be created.
 - [] Entertainer can be added.
 - [] Entertainer can complete required onboarding.
 - [] QR can be generated.
 - [] QR resolves correctly.
 - [] Guest can tip without creating an account.
 - [] Payment provider checkout works.
 - [] Payment webhook is verified.
 - [] Duplicate webhooks are harmless.
 - [] Payment is recorded exactly once.
 - [] Ledger balances.
 - [] Split rules are applied correctly.
 - [] Entertainer payable balance is correct.
 - [] Venue payable balance is correct.
 - [] Platform revenue is correct.
 - [] Payout can be initiated.
 - [] Payout retries safely.
 - [] Failed payouts retain the liability.
 - [] Guest receives confirmation.
 - [] Entertainer sees transaction.
 - [] Venue sees transaction.
 - [] Reconciliation works.
 - [] Audit trail exists.
 - [] Critical errors generate alerts.
 - [] Security review is completed.
 - [] Payment/compliance structure is validated.
 - [] Pilot venue successfully completes real transactions.
-

66. Final Product Definition

The product is **not simply a digital version of spraying money.**

It is a financial infrastructure layer for entertainment.

The first experience is intentionally tiny:

Scan → Tip → Pay → Done.

Behind that simple experience is a serious system:

Payment → Attribution → Double-entry Ledger → Split → Payout → Reconciliation.

The initial Nigerian nightlife product proves the behavior.

The underlying infrastructure becomes the long-term company.

Start with the clubs. Build the rail underneath.